Integrantes: Samuel Suira, Dereck Martínez, Cristian Valdés
Fecha: 29 de agosto de 2026
Docente: Alexis Flores Franco

Justificación escrita de los patrones diagramados

Patrón: Maestro-Esclavo

Caso de uso elegido

Procesamiento distribuido de un lote de facturas. El sistema recibe un lote de 20 facturas y debe entregar un reporte con el subtotal, el ITBMS del 7 % y el total con impuesto. Un componente maestro parte el lote en cuatro sublotes de cinco facturas y le entrega uno a cada nodo de cálculo. Los cuatro nodos trabajan al mismo tiempo: cada uno suma los montos que le tocaron, les aplica el impuesto y le devuelve al maestro su resultado parcial, con la cantidad de facturas procesadas, su subtotal y su ITBMS. El maestro junta esos cuatro resultados y arma el reporte final del lote.

¿Por qué este patrón encaja con el caso?

Escogimos este patrón porque cada factura se calcula sola: para sacarle el ITBMS a una no hace falta saber nada de las demás, y esa independencia es justo la condición que el patrón pide para poder repartir el trabajo. Además el trabajo es parejo, todas las facturas cuestan más o menos lo mismo de calcular, así que partir el lote en cuatro pedazos iguales aprovecha bien los nodos y el tiempo total baja casi en la misma proporción en que los agregamos. Y como al final el negocio necesita un solo reporte del lote completo, tiene sentido que exista un componente que reparta al inicio y junte al final; sumar en cualquier orden da el mismo total, así que tampoco importa cuál nodo termine primero.

Principal desventaja en este caso y cómo la mitigaríamos

Todo pasa por el maestro, y ahí está el punto débil: si se cae a media corrida se pierde el avance de los cuatro nodos. A eso se suma que recoge los resultados en el mismo orden en que los repartió, entonces un nodo lento deja esperando a los demás aunque ya hayan terminado lo suyo; con miles de facturas el más lento termina marcando el ritmo de todo el lote. Lo mitigaríamos cortando el lote en más pedazos que nodos y dejándolos en una cola de trabajo, para que el que se desocupa tome el siguiente en vez de quedarse quieto y el reparto se ajuste solo a la velocidad real de cada máquina. También recogeríamos cada resultado apenas llega y no en el orden en que se enviaron, que en Java se hace con CompletionService en lugar de recorrer los Future uno por uno. Por último, guardaríamos el avance del lote fuera del maestro y le pondríamos un tiempo límite a cada sublote, de modo que si un nodo se cuelga ese pedazo se le reasigne a otro y no haya que empezar de cero.

Patrón: Filtro de Tubería (Pipe-Filter)

Caso de uso elegido

Sistema de análisis de logs de un servidor. Las líneas del log entran a una tubería de tres filtros encadenados. El primero descarta las que solo informan (INFO y DEBUG) y deja pasar únicamente las de ERROR y WARN. El segundo agrupa las que quedaron por tipo de error y cuenta cuántas veces apareció cada uno. El tercero convierte esa cuenta en el reporte de texto que lee el equipo de soporte. Entre cada par de filtros viaja un formato definido: entra una lista de líneas de log (List<LineaLog>), sale esa misma lista ya filtrada, después una lista de incidencias con su conteo (List<Incidencia>) y al final el reporte armado como texto (String).

¿Por qué este patrón encaja con el caso?

Escogimos este patrón porque el trabajo ya viene ordenado en pasos que no se pueden invertir: no se pueden contar errores que todavía no se han separado, ni escribir el reporte sin tener antes el conteo. Cada filtro depende solo del formato que recibe y no de quién se lo entregó, así que los podemos probar por separado, cambiar uno o meter uno nuevo en el medio sin tocar los demás; si más adelante nos piden ocultar datos personales, agregamos ese filtro entre el segundo y el tercero y el resto queda igual. La clase Tuberia es la única que sabe el orden, así que todo el acoplamiento queda concentrado en un solo lugar.

Principal desventaja en este caso y cómo la mitigaríamos

Cada filtro tiene que esperar a que el anterior termine con toda la lista antes de arrancar, y de paso va creando una lista nueva en memoria. Con nueve líneas eso no importa, pero con un log de millones las esperas se van sumando, hay que sostener la colección completa en memoria y el reporte recién sale cuando terminó el último filtro, así que tampoco sirve para avisar de un problema en el momento en que ocurre. Lo mitigaríamos pasando las líneas de una en una en vez de la lista entera, con Stream o con una cola entre filtros, para que cada línea recorra toda la tubería sin esperar a las demás y la memoria deje de crecer con el tamaño del log. Además cortaríamos el log en ventanas de tiempo, por ejemplo de un minuto, para ir sacando reportes parciales durante el día en lugar de uno solo al final.

Nota comparativa

Ninguno de los dos casos admite el patrón contrario. Las facturas se pueden repartir justamente porque ninguna necesita el resultado de otra, mientras que los tres pasos del análisis de logs están encadenados y el segundo trabaja sobre lo que le dejó el primero. Armar el lote de facturas como una tubería sería poner en fila un trabajo que puede hacerse al mismo tiempo, y repartir los filtros del log entre varios esclavos haría que cada uno recibiera datos que todavía no están listos.

Comprobación con los programas de ejemplo

Compilamos y ejecutamos en OpenJDK 21 los dos programas adjuntos a la tarea. En MaestroEsclavoFacturas el maestro repartió las 20 facturas entre cuatro esclavos y consolidó un subtotal de 2 543.50, un ITBMS de 178.05 y un total de 2 721.55; el orden en que los esclavos imprimen cambia entre corridas, pero el total siempre es el mismo, lo que confirma que las subtareas son independientes. En PipeFilterLogs la tubería redujo las 9 líneas a 6 de nivel ERROR y WARN, las agrupó en 3 tipos de error y devolvió el reporte de incidencias; aquí la salida es idéntica en cada corrida, porque los filtros corren siempre en el mismo orden.